

MySQL – Beágyazott lekérdezések (Subqueries)

Szint: Kezdő–Középhaladó | **Téma:** Egyszerű beágyazott lekérdezések

1. Mi az a beágyazott lekérdezés?

A **beágyazott lekérdezés** (angolul: *subquery* vagy *nested query*) egy olyan **SELECT** utasítás, amely egy másik SQL utasítás belsejében szerepel. A belső lekérdezés mindig zárójelben van, és a külső lekérdezés felhasználja annak eredményét.

```
KÜLSŐ LEKÉRDEZÉS (  
    BELSŐ LEKÉRDEZÉS  ← ez fut le először  
)
```

💡 **Fontos:** A belső lekérdezés **mindig hamarabb fut le**, mint a külső. Az eredménye átadódik a külső lekérdezésnek.

2. Beágyazott lekérdezés, amely egyetlen értéket ad vissza

Ha a belső lekérdezés **egyetlen cellát** ad vissza (egy sor, egy oszlop), akkor összehasonlító operátorokkal (**=**, **>**, **<**, **>=**, **<=**, **<>**) használhatjuk.

Szintaxis

```
SELECT oszlop1, oszlop2  
FROM tabla  
WHERE oszlop operátor (SELECT egy_ertek FROM tabla2 WHERE feltétel);
```

Példa 1 – Ki a legmagasabb fizetésű alkalmazott?

```
-- Melyik alkalmazott keres annyit, mint a maximális fizetés?  
SELECT nev, fizetes  
FROM alkalmazottak  
WHERE fizetes = (SELECT MAX(fizetes) FROM alkalmazottak);
```

Magyarázat:

- Először lefut: **SELECT MAX(fizetes) FROM alkalmazottak** → pl. visszaad **850000**-t
- Ezután a külső lekérdezés ezt látja: **WHERE fizetes = 850000**

Példa 2 – Átlag feletti értékek

```
-- Kik keresnek az átlagnál többet?  
SELECT nev, fizetes  
FROM alkalmazottak  
WHERE fizetes > (SELECT AVG(fizetes) FROM alkalmazottak);
```

Példa 3 – Más táblából jövő egyedi érték

```
-- Kik dolgoznak abban a városban, ahol a cég székhelye van?  
SELECT nev, varos  
FROM alkalmazottak  
WHERE varos = (SELECT szekhely FROM cég WHERE id = 1);
```

3. Beágyazott lekérdezés, amely értéklístát ad vissza

Ha a belső lekérdezés **több sort** adhat vissza (de mindig csak egy oszlopot), akkor az **IN** vagy **NOT IN** operátorokat kell használni.

3.1 Az **IN** operátor

Az **IN** megvizsgálja, hogy az adott érték **szerepel-e** a belső lekérdezés által visszaadott listában.

Szintaxis

```
SELECT oszlop1, oszlop2  
FROM tabla  
WHERE oszlop IN (SELECT egy_oszlop FROM tabla2 WHERE feltétel);
```

Példa 4 – Adott részlegen dolgozók

```
-- Kik dolgoznak az IT vagy a Marketing részlegen?  
SELECT nev, reszleg_id  
FROM alkalmazottak  
WHERE reszleg_id IN (  
    SELECT id  
    FROM reszlegek  
    WHERE nev IN ('IT', 'Marketing')  
);
```

Magyarázat:

1. Belső lekérdezés visszaadja pl.: (3, 7) (az IT és Marketing részleg azonosítói)
2. Külső lekérdezés: **WHERE reszleg_id IN (3, 7)**

Példa 5 – Legalább egy rendelést leadó ügyfelek

```
-- Mely ügyfelek adtak le legalább egy rendelést?  
SELECT nev, email  
FROM ugyfelek  
WHERE id IN (  
    SELECT DISTINCT ugyfel_id  
    FROM rendelesek  
);
```

💡 A **DISTINCT** kulcsszó nem kötelező az **IN**-nel, de jó gyakorlat, mert csökkenti a belső lekérdezés eredményének méretét.

3.2 A **NOT IN** operátor

A **NOT IN** megvizsgálja, hogy az adott érték **NEM szerepel-e** a belső lekérdezés által visszaadott listában – tehát a kizárásra szolgál.

Szintaxis

```
SELECT oszlop1, oszlop2  
FROM tabla  
WHERE oszlop NOT IN (SELECT egy_oszlop FROM tabla2 WHERE feltétel);
```

Példa 6 – Rendelést még nem leadó ügyfelek

```
-- Mely ügyfelek NEM adtak le még egyetlen rendelést sem?  
SELECT nev, email  
FROM ugyfelek  
WHERE id NOT IN (  
    SELECT DISTINCT ugyfel_id  
    FROM rendelesek  
);
```

Példa 7 – Nem elérhető termékek kizárása

```
-- Csak az elérhető kategóriák termékeit listázza  
SELECT nev, ar  
FROM termek  
WHERE categoria_id NOT IN (  
    SELECT id  
    FROM kategoriak  
);
```

```
WHERE aktiv = 0  
);
```

4. ⚠ Figyelem: NULL értékek és a NOT IN

A **NOT IN** csendben hibás eredményt adhat, ha a belső lekérdezés eredményei között **NULL** érték is szerepel!

```
-- Ha a rendelések táblában bármely ugyfel_id NULL, ez üres eredményt ad vissza!  
SELECT nev FROM ugyfelek  
WHERE id NOT IN (SELECT ugyfel_id FROM rendelések);
```

Megoldás: Szűrjük ki a **NULL** értékeket a belső lekérdezésből:

```
SELECT nev FROM ugyfelek  
WHERE id NOT IN (  
    SELECT ugyfel_id  
    FROM rendelések  
    WHERE ugyfel_id IS NOT NULL -- ← ezt mindig adjuk hozzá NOT IN esetén!  
);
```

⚠ **Szabály:** **NOT IN** használatakor **mindig** adjunk **WHERE ... IS NOT NULL** feltételt a belső lekérdezéshez!

5. Beágyazott lekérdezés a SELECT listában

A beágyazott lekérdezés nem csak a **WHERE** részben szerepelhet, hanem a **SELECT** mezőlistájában is – ilyenkor **mindig egyetlen értéket kell visszaadnia**.

```
-- Minden termék mellé kiírja, hogy hány rendelés tartalmazza  
SELECT  
    nev,  
    ar,  
    (SELECT COUNT(*) FROM rendeles_tetelek rt WHERE rt.termek_id = t.id) AS  
    rendelések_szama  
FROM termek t;
```

6. Beágyazott lekérdezés a FROM részben

A belső lekérdezés „virtuális táblát” is alkothat, amelyet a külső lekérdezés **FROM**-ként kezel. Ilyenkor **alias nevet kell adni** neki.

```
-- Az átlagos rendelési összeg feletti rendelések száma városonként
SELECT varos, db
FROM (
    SELECT varos, COUNT(*) AS db
    FROM rendelesek
    GROUP BY varos
) AS varosonkenti_adatok
WHERE db > 10;
```

7. Összefoglaló táblázat

Visszaadott értékek száma	Használható operátorok	Példa
Egyetlen érték (1 sor, 1 oszlop)	=, <>, >, <, >=, <=	WHERE ar = (SELECT MAX(ar) ...)
Több sor, egy oszlop	IN, NOT IN	WHERE id IN (SELECT id ...)
Több sor, több oszlop	EXISTS, NOT EXISTS	(haladó téma)

8. Jó gyakorlatok

☑ Ajánlott	✗ Kerülendő
NOT IN esetén mindig szűrd ki a NULL-okat	NOT IN használata NULL-t tartalmazó listával
Adj alias nevet a FROM-ban szereplő beágyazott lekérdezésnek	Alias nélküli virtuális tábla
Teszteld a belső lekérdezést önállóan is	Közvetlenül komplex beágyazást írni
Használj DISTINCT-et, ha lehetséges duplikátum	Szükségtelen duplikátumok a listában

9. Gyors referencia

```
-- Egyetlen értékes beágyazás (=, >, < stb.)
SELECT * FROM tabla
WHERE mezo = (SELECT MAX(mezo) FROM tabla2);

-- IN - szerepel a listában
SELECT * FROM tabla
WHERE mezo IN (SELECT mezo FROM tabla2 WHERE feltétel);

-- NOT IN - NEM szerepel a listában (NULL biztos verzió)
SELECT * FROM tabla
WHERE mezo NOT IN (
    SELECT mezo FROM tabla2
    WHERE mezo IS NOT NULL
```

```
);

-- Beágyazás a SELECT listában
SELECT nev, (SELECT COUNT(*) FROM t2 WHERE t2.fk = t1.id) AS db
FROM tabla t1;

-- Beágyazás a FROM-ban (virtuális tábla)
SELECT * FROM (
    SELECT mezo, COUNT(*) AS db FROM tabla GROUP BY mezo
) AS al_tabla
WHERE db > 5;
```
